

Lista de Exercícios 07 - Árvore Binária de Busca

Professora: *Marta Noronha*

Disciplina: *Laboratório de Algoritmos e Estrutura de Dados II*

Data: 26/06/2026 Valor:

Requisitos

1. Todos os programas deverão ser desenvolvidos na linguagem de programação Java.
2. Essas práticas poderão ser desenvolvidas em grupos de, no máximo, dois integrantes.
3. Cópias, se existirem, serão encaminhadas ao colegiado de coordenação didática do curso.
4. As saídas esperadas, cadastradas no VERDE pelo professor, foram geradas empregando-se:
`System.out.println()`.
5. Em cada submissão, enviar apenas um arquivo (.java). A regra será necessária para a submissão de exercícios no VERDE e no identificador de plágios utilizado na disciplina.
6. A resolução (código) de cada exercício deverá ser submetida ao VERDE.
7. A execução do código submetido será realizada automaticamente pelo VERDE, mas o código será analisado e validado pelo professor.

Criação da classe Jogo

O conjunto de dados utilizado neste exercício foi disponibilizado em Kaggle.

Para essa atividade, opcionalmente, você pode utilizar a classe Jogo especificada e desenvolvida nas listas anteriores.

Implemente uma classe Jogo com os seguintes atributos declarados com modo de acesso privado:

- `rank` (int): classificação (posição) do jogo em relação aos demais.
- `nome do jogo` (String): o nome do jogo.
- `plataforma` (String): Onde o jogo pode ser executado (Wii, DS, entre outros).
- `ano` (int): ano de lançamento.
- `gênero` (String): esportes, corrida, entre outros.
- `editora` (String): empresa que disponibiliza o jogo (produtora).
- `NA_Vendas` (double): Vendas do jogo na América do Norte.
- `EU_Vendas` (double): Vendas do jogo na Europa.
- `JP_Vendas` (double): Vendas do jogo no Japão.
- `Outras_Vendas` (double): Vendas do jogo em outros lugares.
- `Vendas_Global` (double): Vendas global do jogo.

A classe também deve conter, obrigatoriamente, ao menos, dois construtores (1. *padrão (default)*; 2. (*nome do jogo/plataforma/ano*), e os métodos *gets*, *sets*, métodos *clone()*, *ler()*, *imprimir()* e *toString()*.

O método *clone()* deve retornar um objeto da mesma classe e contendo os mesmos valores de atributos do atual objeto analisado (**vide exemplo do método clone no slide "Unidade 0 - Nivelamento - Ponteiros**

(C/C++) e Referências (Java)" postado na disciplina teórica). **Não é permitido usar a interface Cloneable para esta finalidade.**

O método `ler()` deve receber cada linha do arquivo como **parâmetro**. Cada linha possui os dados de atributos de cada objeto do tipo `Jogo`. O método deve armazenar os valores contidos em cada linha nos atributos de cada objeto que foi instanciado.

O método `toString()` deve ser implementado para permitir a impressão de um objeto da classe `Jogo`, sem necessidade de invocação do método `imprimir()`. Isso é feito pela sobrescrita do método `toString()`, um método implementado na super classe **Object**. Esse método retorna, por padrão, uma *String* contendo as informações do objeto, o qual deve ser implementado para apresentar as informações conforme o modelo indicado no fim deste documento. [Clique AQUI para ver o modelo](#).

Em Java, a classe **Object** define os comportamentos mínimos que todos os objetos possuem. Um desses métodos é o **toString()**, cujo propósito é fornecer uma representação textual do objeto. Por padrão, a implementação de `toString()` na classe `Object` **retorna uma string** contendo o nome da classe seguido de um código hash, mas é comum que os desenvolvedores sobrescrevam esse método para fornecer uma representação mais significativa, geralmente exibindo os valores dos atributos do objeto.

Todas as classes em Java, inclusive a que contém o método `main`, herdam da classe `Object`. Dessa forma, ao sobrescrever o método `toString()`, o desenvolvedor redefine a forma como o objeto é convertido em string, permitindo uma representação customizada e mais informativa.

O método `imprimir()` é um método público sem retorno, implementado na classe `Jogo`, que imprime na saída padrão a representação textual de um objeto do tipo `Jogo`. Para isso, o método deve simplesmente chamar o método de impressão informando, como parâmetro, o objeto por meio da palavra-chave *this*.

```
public void imprimir() {  
    System.out.println(this);  
}
```

Aqui, `println(Object)` recebe o próprio objeto (*this*), chama automaticamente o método sobrescrito `toString()` e exibe a string formatada conforme o modelo exigido.

Os métodos que não obedecem a forma como devem ser implementados, conforme descrição acima, serão penalizados ainda que o envio no Verde contabilize 100% de acerto. Isso será feito porque o desenvolvedor deve seguir atentamente as regras de implementação de um método, quando houver, considerando os parâmetros e o tipo de retorno que um método deve possuir quando estes são solicitados. Em caso de dúvidas, consulte a Professora.

Após a criação da classe, deve ser criado um mecanismo para processamento de uma entrada de dados armazenados no arquivo *jogos.txt*.

Seu programa deve ler um arquivo-texto chamado **"jogos.txt"** que, no VERDE, localiza-se na pasta **"/tmp"**.

Nesta tarefa, devem ser realizadas as seguintes atividades:

- Parte 1: Leitura de um arquivo-texto e preenchimento de um vetor de jogos.
- Parte 2: Pesquisa de jogos armazenados no vetor com inclusão de jogos selecionados na árvore.

- Parte 3: Pesquisa de informações na árvore

Parte 1: Leitura de arquivo e armazenamento de informações em um vetor

Você deve ler o arquivo-texto "jogos.txt" e preencher um vetor que armazena objetos do tipo Jogo. A cada linha lida do arquivo, um jogo deve ser instanciado. As informações lidas sobre o jogo em cada linha do arquivo devem ser armazenadas em cada objeto instanciado, sendo o objeto inserido no vetor de objetos.

Cada uma das linhas de entrada apresenta os dados de um jogo, separados pelo símbolo "|". Os dados possuem, em ordem, as seguintes informações

- rank (int): classificação (posição) do jogo em relação aos demais.
- nome do jogo (String): o nome do jogo.
- plataforma (String): Onde o jogo pode ser executado (Wii, DS, entre outros).
- ano (int): ano de lançamento.
- gênero (String): esportes, corrida, entre outros.
- editora (String): empresa que disponibiliza o jogo (produtora).
- NA_Vendas (double): Vendas do jogo na América do Norte.
- EU_Vendas (double): Vendas do jogo na Europa.
- JP_Vendas (double): Vendas do jogo no Japão.
- Outras_Vendas (double): Vendas do jogo em outros lugares.
- Vendas_Global (double): Vendas global do jogo.

Importante: O caractere "|" é utilizado para expressões regulares. Para dividir uma string por "|", utilize a seguinte sequência: "\ |".

A última linha do arquivo "jogos.txt" é vazia.

Parte 2: Pesquisa de jogos armazenados no vetor com inclusão de jogos selecionados na árvore.

Nesta parte do trabalho, deve ser instanciada uma árvore de objetos da classe Jogos.

Após a leitura do arquivo, na primeira parte do trabalho, o programa deve ler informações da entrada (pub.in) e realizar inserções em uma árvore binária de busca. No *pub.in*, os dados de cada jogo são separados por ponto-e-vírgula (;).

Cada linha da segunda parte contém, em ordem, as seguintes informações:

- nome do jogo;
- ano;
- editora;
- plataforma.

Para cada jogo mostrado em cada linha lida no *pub.in*, uma pesquisa no vetor de jogos deve ser realizada. Se o jogo pesquisado estiver no vetor de jogos (criados na Parte 1) então o jogo deve ser inserido na árvore de jogos.

O processamento da entrada padrão, nesta parte, deve acontecer até que o código encontre a palavra FIM.

Parte 3: Pesquisa de informações na árvore

Após a criação da árvore, na segunda parte do trabalho, o programa deve ler informações da segunda parte da entrada (pub.in).

Cada jogo (contido na entrada) deve ser pesquisado na árvore (criada na parte 2). Se o jogo estiver na árvore, deve ser exibido o caminho até o jogo, seguido da palavra “SIM”.

Caso o jogo não seja encontrado, deve ser exibido o caminho pesquisado na árvore, seguido da palavra “NÃO”.

Cada jogo encontrado no caminho deve ter sua saída padrão obedecendo o seguinte formato (*não incluir chaves*):

[Nome do jogo] [Plataforma] [Vendas global] {Rank}. {Gênero}. {Editora}. Mais vendido:{País ou região mais vendido}.

Exemplos:

[Zero no Tsukaima: Shou-akuma to Harukaze no Concerto] [XB] [1.72] 1040. Sports. Electronic Arts. Mais vendido: NA_Vendas.

- [NHL Blades of Steel 2000] [PS3] [0.63] 3201. Misc. Activision. Mais vendido: NA_Vendas.

- [Rome: Total War - Gold Edition] [PS2] [0.62] 3265. Sports. Play It. Mais vendido: NA_Vendas.

- [Reel Fishing II] [X360] [0.13] 9647. Shooter. D3Publisher. Mais vendido: NA_Vendas.

- [Reversal Challenge] [Wii] [0.61] 3303. Sports. Activision. Mais vendido: NA_Vendas.

- SIM

OBS: Após finalizar o caminho, se o jogo foi encontrado imprima SIM. Caso contrário, imprima NÃO.

Para implementação dos mecanismos de comparação de menor ou maior para a inserção correta na árvore, utilize os seguintes critérios:

- Nome do jogo (crescente);
- Plataforma (crescente);
- Ano (decrecente).

Implementar uma função de igualdade que verifica se todos os atributos abaixo de dois jogos são iguais:

- nome do jogo;
- ano;
- editora;
- plataforma.

Caso não sejam iguais, fazer a comparação usando o método para comparação de menor ou maior para a pesquisa na árvore.

OBS: Para os métodos de verificação de menor ou maior, use a função *compareToIgnoreCase*.

O processamento da entrada padrão, nesta parte, deve acontecer até que o código encontre a palavra FIM.

Ao final, devem ser impressas as informações sobre as quantidades de jogos encontrados e não encontrados no formato:

Jogos Encontrados: *quantidade*

Jogos Não Encontrados: *quantidade*